

# Space Invaders Bug Analysis

<https://gemini.google.com/app/582de7446347d8b8>

User prompt: I am working on a college lab project involving a partially implemented Space Invaders game using Python and Pygame. I have cloned the original repository and am uploading the project files, including the README and source code. Your first task is to analyze the existing project and identify bugs before making any changes. Follow these instructions carefully: Read the README.md completely and understand the four required tasks: Task 1: Refine collision detection. Task 2: Implement a proper Game Over screen. Task 3: Add a replay option with Easy, Medium, and Hard difficulty settings. Task 4: Add sound effects for firing, destroying an enemy, and game over. Inspect every uploaded Python file and understand how the game currently works. Trace the program execution from its actual entry point through the game engine, player, enemies, and bullets. Identify existing bugs, incomplete implementations, logical errors, potential crashes, and inconsistencies between the README and the actual source code. Pay particular attention to: Bullet-enemy collision detection and whether collisions can be missed. Whether bullets or enemies are accidentally skipped or processed more than once. Player movement, shooting, and enemy movement. Enemy firing and player collision. Score calculation and display. Game loop termination and event handling. Missing assets, dependencies, imports, or incorrect file paths. Whether the game can start and run using the instructions in the README. Whether the actual entry point is main.py or game.py. Do not assume either one without checking the files. For every suspected bug, explain: The file and function involved. The relevant code or logic responsible. The conditions under which the bug occurs. The expected behavior versus the current behavior. The severity and whether it is confirmed from the source code or merely a potential issue. Separate confirmed bugs from potential improvements. Do not invent bugs or claim to have executed the game unless you actually have execution capability and have tested it. Do not edit, rewrite, create, or delete any files yet. Do not install packages, add features, or apply fixes at this stage. Do not assume that the four tasks are the only bugs in the original repository. Report any additional issues you find, but keep the required lab tasks clearly identified. At the end, provide: A concise summary of the current state of the game. A prioritized list of confirmed and suspected bugs. A list of files likely to require changes for each of the four tasks. The exact commands I should run locally to install the dependencies and launch the original game, based on the actual repository structure. Any missing information or files you need before you can complete the analysis. This is an analysis-only phase. Wait for my review before suggesting or applying code changes. I want to understand the original bugs and preserve the original project before beginning the fixes.

## Response: Project State & Entry Point Analysis

- **actual Entry Point:** main.py. Running python main.py starts the Pygame application loop, which initializes Pygame, creates a window (600 × 700), and calls methods on GameEngine.
- **Current Game State:** The core engine mechanics (player movement, shooting, enemy grid movement, basic collision checks, score updating, and enemy shooting) are partially functional. However, the game lacks full task implementations (proper Game Over UI, Replay menu, Sound effects), contains a critical iteration bug in collision handling, and exhibits several edge-case logical bugs.

## Analysis of Required Lab Tasks

### Task 1: Refine Collision Detection

- **File & Function:** game/game\_engine.py → GameEngine.update()
- **Logic Responsible:**

```
for bullet in self.player_bullets:
    for enemy in self.enemy_grid.alive_enemies():
        if bullet.rect().colliderect(enemy.rect()):
            enemy.alive = False
            self.player_bullets.remove(bullet)
            self.score += 1
            break
'''[cite: 6]
```

- **Conditions Under Which Bug Occurs:** When a player bullet collides with an enemy.
- **Expected vs Current Behavior:**
  - *Current:* Modifying self.player\_bullets by calling .remove(bullet) while iterating over self.player\_bullets causes Python's list iterator index to shift. The element immediately following the removed element is skipped in the loop. If multiple bullets are active in close proximity or consecutive frames, bullets get skipped during evaluation, allowing them to pass straight through enemies.
  - *Expected:* Collision checks should reliably process every active bullet and enemy without skipping elements during iteration.
- **Severity: Confirmed (High Severity / Game Defect).**

### Task 2: Implement Game Over Screen

- **File & Function:** game/game\_engine.py → GameEngine.render() & update()
- **Logic Responsible:**

```
if self.game_over and not getattr(self, "_game_over_logged", False):
    print("Game over! Final score:", self.score)
    self._game_over_logged = True
'''[cite: 6]
```

- **Conditions Under Which Bug Occurs:** Player gets hit by an enemy bullet or enemies reach player.y.

- **Expected vs Current Behavior:**

- *Current:* Game simply freezes frame rendering (since `update()` returns early when `self.game_over` is `True`) and prints a message to the console terminal once.
- *Expected:* Render a graphical Game Over overlay on screen showing the final score, waiting gracefully for user input.

- **Severity: Confirmed (Incomplete Feature / Missing UI).**

### Task 3: Add Replay Option & Difficulty Settings

- **File & Function:** `game/game_engine.py`, `game/enemy.py`, `main.py`

- **Logic Responsible:** `GameEngine.__init__()` and `EnemyGrid.__init__()` currently have fixed speed (1.5) and fire chance (0.01) with no state-reset or difficulty parameterization.

- **Conditions Under Which Bug/Limitation Occurs:** Player reaches Game Over and wants to restart without re-executing the process.

- **Expected vs Current Behavior:**

- *Current:* Game halts permanently upon Game Over; user must close window and relaunch.
- *Expected:* Provide input/menu options (e.g., Easy, Medium, Hard) to restart with adjusted speeds/fire rates or exit.

- **Severity: Confirmed (Missing Feature).**

### Task 4: Add Sound Effects

- **File & Function:** `game/game_engine.py`

- **Logic Responsible:** No audio assets loaded, initialized, or triggered in shooting, enemy hit, or game over routines.

- **Conditions Under Which Bug/Limitation Occurs:** Normal gameplay actions.

- **Expected vs Current Behavior:**

- *Current:* Game is completely silent.
- *Expected:* Trigger sound effects for firing, destroying an enemy, and game over.

- **Severity: Confirmed (Missing Feature).**

### Additional Confirmed Bugs & Logical Issues

#### 1. Unbounded Enemy Firing Rate (Framerate & Grid-Size Dependent)

- **File & Function:** `game/game_engine.py` → `GameEngine.update()`

- **Code Responsible:**

```
for enemy in self.enemy_grid.alive_enemies():
    if random.random() < self.enemy_fire_chance:
        bullet_x = enemy.x + enemy.width // 2
        self.enemy_bullets.append(Bullet(bullet_x, enemy.y + enemy.height, direction=1))
```[cite: 6]
```

- **Condition:** Every frame, every alive enemy individually rolls a 1% chance to fire.
- **Behavior:** With 32 enemies active at 60 FPS, an average of 19.2 bullets spawn *per second*, overwhelming the screen with an impassable wall of lasers. Furthermore, top-row enemies fire straight through bottom-row enemies.
- **Severity: Confirmed (High Severity / Design Defect).**

#### 2. Inconsistent README Description vs Implementation

- **File:** `README.md`
- **Discrepancy:** The README states: "*This project is a terminal-based Space Invaders clone using Pygame*".
- **Reality:** The project uses full Pygame GUI graphics via `pygame.display.set_mode()`. It is a GUI application, not a terminal/text UI application.
- **Severity: Confirmed Documentation Inconsistency.**

### Potential / Suspected Issues

#### 1. High Movement Speed Fast-Bullet Tunneling (Potential)

- **File:** `game/bullet.py` → `Bullet.move()`
- **Condition:** Bullets move 8 pixels/frame.
- **Behavior:** If enemy size or bounding box thickness is narrow, fast moving bullets checked solely via `static colliderect()` might jump across thin targets between frames.

Prioritized Bug & Task List

1. **[Critical] Fix List Iteration Collision Bug (Task 1):** Prevent list mutation during iteration in `GameEngine.update()`.
2. **[High] Fix Enemy Firing Rate Balance:** Restrict enemy firing to bottom-most row enemies and add a global fire cooldown/limiter.
3. **[Medium] Implement Game Over Screen (Task 2):** Render game-over UI and hold event loop state.
4. **[Medium] Implement Replay & Difficulty Selection (Task 3):** Add state resetting logic and difficulty parameter mapping.
5. **[Low] Implement Audio Manager / Sound FX (Task 4):** Load audio assets with fallbacks for missing audio drivers.

Summary of Files Requiring Changes per Task

| Task                        | Files to Modify                                                                      |
|-----------------------------|--------------------------------------------------------------------------------------|
| Task 1: Refine Collisions   | <code>game/game_engine.py</code>                                                     |
| Task 2: Game Over Screen    | <code>game/game_engine.py</code> , <code>main.py</code>                              |
| Task 3: Replay & Difficulty | <code>game/game_engine.py</code> , <code>game/enemy.py</code> , <code>main.py</code> |
| Task 4: Sound Effects       | <code>game/game_engine.py</code> (and new sound assets folder)                       |

Execution & Launch Commands

To install dependencies and run the unmodified base game locally, run the following commands in your terminal:

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Launch the game using the actual entry point
python main.py
```[cite: 1, 2, 3]

---

### Missing Information / Files Needed

1. **Audio Assets (for Task 4):** The project repository does not include audio files (`.wav` or `.ogg`). Sound files fo

Please review this analysis and let me know when you are ready to proceed with generating solutions or code changes!
```

User prompt: The initial source-code analysis is complete. I now want to establish a baseline for the original, unmodified game before implementing any fixes. Guide me through verifying the original game locally. Confirm that the correct entry point is `main.py` and that the dependency installation command is appropriate for this repository. Explain how I can verify that the game launches and that player movement, shooting, enemy movement, enemy firing, and scoring work. Explain how I can reproduce or observe the reported collision bug, without modifying the source code. Identify which behaviors cannot reliably be tested without changing the code, and distinguish these from bugs that can be observed during normal gameplay. Do not edit any files, install additional packages, or implement any fixes. I will perform the installation, launch, and recording on my own computer. Give me a short baseline-testing checklist and tell me what observations or errors to report back before we begin Task 1. Keep the original source code unchanged. We will implement the four README tasks incrementally, reviewing and testing each task before committing it separately.

Response: Here is the guide and baseline-testing procedure to verify the unmodified Space Invaders game locally on your computer before introducing any code modifications or fixing bugs.

1. Entry Point & Dependency Verification

- **Entry Point Confirmation:**
  - `main.py` is the actual entry point of the application.
  - Running `python main.py` executes the Pygame initialization (`pygame.init()`), creates the window (`pygame.display.set_mode((600, 700))`), instantiates `GameEngine`, and manages the event loop.
  - `game/game_engine.py` is a class library module containing `GameEngine` and is imported by `main.py`. It is **not** meant to be executed directly.

- **Dependency Command:**

- The command specified in the README.md (`pip install -r requirements.txt`) is appropriate and sufficient. It installs pygame, which is the only external dependency listed.

## 2. Manual Gameplay & Verification Steps

To verify that core gameplay mechanics function in the baseline build:

### 1. Launch the Game:

- Run `python main.py` in your terminal.
- A window titled "**Space Invaders - Pygame Version**" ( $600 \times 700$  pixels) with a black background should appear.

### 2. Player Movement:

- Press **Left Arrow** or **A** to move the green player rectangle to the left.
- Press **Right Arrow** or **D** to move the green player rectangle to the right.
- **Boundary check:** Hold either direction key to ensure the player ship stops at the screen edges ( $x = 0$  or  $x = \text{screen\_width} - \text{player\_width}$ ) without moving off-screen.

### 3. Player Shooting:

- Press **Space**. A thin white rectangle (player bullet) should fire upward from the center of the player ship.
- Repeatedly tap **Space** to confirm the shooting rate-limiter works (there is a 15-frame cooldown per bullet).

### 4. Enemy Grid Movement & Drop:

- Observe the white enemy grid marching horizontally across the screen.
- When the rightmost or leftmost enemy reaches the window boundary, the entire grid should reverse direction and drop downward by 20 pixels.

### 5. Enemy Firing & Player Hitbox:

- Observe red bullets descending randomly from the grid.
- Move your ship directly into the line of fire of a red bullet.
- Upon collision, the game engine should set `self.game_over = True`, freeze gameplay updates, and print `Game over! Final score: <score>` once in your terminal console.

### 6. Score Tracking:

- Watch the upper-left corner text ("Score: X").
- Destroy an enemy with a player bullet and verify that the score increments by 1.

## 3. How to Reproduce & Observe the Task 1 Collision Bug

You can observe the collision iteration bug directly through gameplay **without changing any code**:

1. **Setup:** Fire continuously into a dense section of the enemy grid (preferably where two adjacent enemies are moving closely together, or fire two bullets in rapid succession).
2. **Observation:** Notice that when a bullet destroys an enemy, a nearby or trailing bullet may pass directly through another active enemy without destroying it or registering a hit.
3. **Underlying Cause in Code:** In `game/game_engine.py`, the engine modifies `self.player_bullets` by calling `.remove(bullet)` while actively iterating over `self.player_bullets`. This shifts the list indices during loop execution, causing Python's iterator to skip the immediate next bullet in the list.

## 4. Behaviors Testable vs. Untestable Without Code Changes

### Observable / Testable During Normal Gameplay (Unmodified Code)

- Basic player horizontal movement and boundary stopping.
- Player bullet spawning and upward motion.
- Enemy grid side-to-side marching and row drop.
- Random enemy bullet generation and downward motion.

- Basic score increments when a single bullet hits an enemy.
- Terminal print log upon player death or enemy reaching the player baseline.
- **The Task 1 Collision Bug:** Bullets passing through enemies when hits occur on consecutive elements.

### Untestable Without Code Changes (Missing Implementations)

- **Task 2 (Game Over UI):** On-screen graphical Game Over text, overlay, or graceful key press handling does not exist yet (only a terminal `print()` statement exists).
- **Task 3 (Replay Menu & Difficulty):** Restarting the game via key press or selecting Easy/Medium/Hard speeds cannot be tested because no menu or reset logic exists.
- **Task 4 (Sound Effects):** Sound effects cannot be heard because no audio files are loaded or played in the base code.

## 5. Baseline Testing Checklist

Use this checklist during your baseline run:

- [ ] Dependencies installed via `pip install -r requirements.txt`.
- [ ] Game successfully launches with `python main.py`.
- [ ] Player moves left/right (A/D or Arrow keys) and stops at screen edges.
- [ ] Pressing Space fires player bullets upward.
- [ ] Enemies march side-to-side and drop down upon contacting screen borders.
- [ ] Red enemy bullets fire randomly and move downward.
- [ ] Single bullet hit destroys an enemy and updates the score counter.
- [ ] Observed the Task 1 bug (bullets passing through enemies during consecutive or rapid collisions).
- [ ] Player hit by enemy bullet triggers game freeze and console terminal log `Game over! Final score: X`.

## What to Report Back

Once you have performed the baseline check on your local system, report back with:

1. Whether `python main.py` launched cleanly without terminal import/runtime errors.
2. Whether you were able to observe the Task 1 collision bug in gameplay.
3. Confirmation that you are ready to begin **Task 1: Refine Collision Detection**.

*User prompt: We are beginning Task 1 from the README: Refine Collision Detection. The original game has been successfully launched and recorded before any modifications. Do not modify or overwrite my original before-video recording. Implement ONLY Task 1 in the existing project. Task description: "Bullets sometimes pass straight through enemies without registering a hit, especially when firing rapidly or when two enemies are hit close together. Investigate and enhance collision accuracy." Requirements: Inspect the current collision handling in game/game\_engine.py and the Bullet and Enemy classes before making changes. Fix the confirmed issue caused by removing bullets from self.player\_bullets while iterating over that same list. Ensure that: Every active player bullet is processed correctly. A bullet is removed after a successful hit. An enemy is counted as destroyed only once. The score increments exactly once per enemy destroyed. Multiple collisions in the same update do not cause bullets or enemies to be skipped. Bullets that leave the visible screen are cleaned up safely. Existing player movement, shooting, enemy movement, enemy firing, and scoring continue to work. Investigate whether fast-moving bullets can tunnel through enemies between frames. If this is a genuine risk in the current implementation, improve collision accuracy appropriately without unnecessarily complicating the code. Keep the changes focused on Task 1. Do not implement the Game Over screen, replay menu, difficulty settings, sound effects, or unrelated gameplay changes. Do not rebalance enemy firing in this task. Preserve the existing project structure, naming conventions, and coding style wherever practical. Do not rewrite the entire game or introduce unnecessary dependencies. After implementation, explain: The exact bug and its cause. Which files you modified and why. The changes you made and how they prevent missed collisions. Any remaining limitations. Provide a practical testing plan covering a single hit, multiple bullets hitting enemies close together, score accuracy, bullet cleanup, and normal gameplay. Include automated tests if they can be added without unnecessary dependencies or intrusive restructuring. Show the complete updated contents of every modified file, or provide a clear, reviewable diff. Do not claim that tests passed unless you actually executed them. Stop after completing Task 1. Wait for my review and local testing before implementing any other task. I will review your changes and test the game locally before proceeding to Task 2.*

## Response: Deep-Dive Analysis of the Collision Bug & Tunneling Risk

### 1. Confirmed Mutation-During-Iteration Bug

In `game/game_engine.py`, the original collision check was written as:

```
for bullet in self.player_bullets:
    for enemy in self.enemy_grid.alive_enemies():
```

```

        if bullet.rect().colliderect(enemy.rect()):
            enemy.alive = False
            self.player_bullets.remove(bullet)
            self.score += 1
            break
    ```[cite: 6]

    * **Cause:** Python maintains an internal index counter when executing `for bullet in self.player_bullets:`. When `self.`
    * **Impact:** Any bullet immediately following a colliding bullet in `self.player_bullets` skips its collision check for

    ##### 2. Fast-Moving Bullet Tunneling Risk Analysis
    In `game/bullet.py`, bullets move at  $8 \text{ pixels/frame}$  (`speed = 8`, `direction = -1`)[cite: 4].
    * Enemies have height  $30 \text{ pixels}$  (`height = 30`) and spacing  $45 \text{ pixels}$ [cite: 5].
    * Because the bullet displacement ( $8 \text{ px}$ ) is smaller than the enemy height ( $30 \text{ px}$ ), standard bounding b
    * **Conclusion on Tunneling:** Discrete step-by-step frame bounding box checks (`colliderect`) are mathematically suffic

    ---

    ### Files Modified & Summary of Changes

    Only `game/game_engine.py` needed modification for Task 1[cite: 6]. No changes were made to `player.py`, `enemy.py`,

    ##### Changes Made to `game/game_engine.py`:
    1. **Separated Collision Detection from List Cleanup:**
        * Track destroyed enemies and spent bullets in temporary sets (`bullets_to_remove` and `destroyed_enemies`)[cite: 6].
        * Perform all collision evaluations over immutable snapshot iterations (or full loops) without mutating `self.player_
    2. **Guaranteed Atomic Operations:**
        * Ensure each bullet can only destroy one enemy per frame.
        * Ensure each enemy is marked dead (`enemy.alive = False`) and scored (`self.score += 1`) exactly once[cite: 6].
    3. **Safe Cleanup:**
        * Clean up spent bullets in a single list comprehension after all checks conclude[cite: 6].

    ---

```

### ### Complete Updated File

Here is the complete updated content for `game/game\_engine.py`:

```

```python
import pygame
import random
from .player import Player
from .enemy import EnemyGrid
from .bullet import Bullet

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.player = Player(width // 2 - 20, height - 50, 40, 20)
        self.enemy_grid = EnemyGrid(width)

        self.player_bullets = []
        self.enemy_bullets = []
        self._shoot_cooldown = 0
        self.enemy_fire_chance = 0.01

        self.score = 0
        self.font = pygame.font.SysFont("Arial", 30)
        self.game_over = False

    def handle_event(self, event):
        if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
            if self._shoot_cooldown <= 0:
                bullet_x = self.player.center_x() - 2
                self.player_bullets.append(Bullet(bullet_x, self.player.y, direction=-1))
                self._shoot_cooldown = 15

```

```

def handle_input(self):
    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT] or keys[pygame.K_a]:
        self.player.move(-self.player.speed, self.width)
    if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
        self.player.move(self.player.speed, self.width)

def update(self):
    if self.game_over:
        return

    if self._shoot_cooldown > 0:
        self._shoot_cooldown -= 1

    self.enemy_grid.move()

    for enemy in self.enemy_grid.alive_enemies():
        if random.random() < self.enemy_fire_chance:
            bullet_x = enemy.x + enemy.width // 2
            self.enemy_bullets.append(Bullet(bullet_x, enemy.y + enemy.height, direction=1))

    for bullet in self.player_bullets:
        bullet.move()
    for bullet in self.enemy_bullets:
        bullet.move()

    # Clean up off-screen bullets
    self.player_bullets = [b for b in self.player_bullets if not b.off_screen(self.height)]
    self.enemy_bullets = [b for b in self.enemy_bullets if not b.off_screen(self.height)]

    # --- TASK 1: Refined Collision Detection ---
    bullets_to_remove = set()
    alive_enemies = self.enemy_grid.alive_enemies()

    for bullet in self.player_bullets:
        for enemy in alive_enemies:
            if enemy.alive and bullet.rect().colliderect(enemy.rect()):
                enemy.alive = False
                bullets_to_remove.add(bullet)
                self.score += 1
                break # This bullet hit an enemy; stop checking this bullet against other enemies

    # Safely remove collided bullets after loop completes
    if bullets_to_remove:
        self.player_bullets = [b for b in self.player_bullets if b not in bullets_to_remove]

    # Check enemy bullet hits against player
    for bullet in self.enemy_bullets:
        if bullet.rect().colliderect(self.player.rect()):
            self.game_over = True
            break

    if self.enemy_grid.reached_bottom(self.player.y):
        self.game_over = True

def render(self, screen):
    pygame.draw.rect(screen, GREEN, self.player.rect())

    for enemy in self.enemy_grid.alive_enemies():
        pygame.draw.rect(screen, WHITE, enemy.rect())

    for bullet in self.player_bullets:
        pygame.draw.rect(screen, WHITE, bullet.rect())
    for bullet in self.enemy_bullets:
        pygame.draw.rect(screen, RED, bullet.rect())

    score_text = self.font.render(f"Score: {self.score}", True, WHITE)
    screen.blit(score_text, (10, 10))

    if self.game_over and not getattr(self, "_game_over_logged", False):
        print("Game over! Final score:", self.score)
        self._game_over_logged = True

```

```

```[cite: 6]

```

```

---

### Code Diff (`game/game_engine.py`)

```diff
@@ -62,13 +62,17 @@
-     # NOTE: this removes a bullet from player_bullets while iterating
-     # directly over that same list. Python skips the element right
-     # after a removed one, so when two enemies are hit on the same
-     # frame the second collision can be missed - the bullet appears
-     # to pass straight through. See Task 1 in the README.
-     for bullet in self.player_bullets:
-         for enemy in self.enemy_grid.alive_enemies():
-             if bullet.rect().colliderect(enemy.rect()):
-                 enemy.alive = False
-                 self.player_bullets.remove(bullet)
-                 self.score += 1
-                 break
+     # --- TASK 1: Refined Collision Detection ---
+     bullets_to_remove = set()
+     alive_enemies = self.enemy_grid.alive_enemies()
+
+     for bullet in self.player_bullets:
+         for enemy in alive_enemies:
+             if enemy.alive and bullet.rect().colliderect(enemy.rect()):
+                 enemy.alive = False
+                 bullets_to_remove.add(bullet)
+                 self.score += 1
+                 break # Bullet hit an enemy; proceed to next bullet
+
+     if bullets_to_remove:
+         self.player_bullets = [b for b in self.player_bullets if b not in bullets_to_remove]
```[cite: 6]

---

### Remaining Limitations

* Enemy Firing Balancing: As required, enemy firing mechanics remain unadjusted for Task 1[cite: 6].
* UI & Audio: Game Over screen, replay/difficulty options, and audio effects remain unimplemented until Tasks 2-4[ci

---

### Practical Manual Testing Plan

Run `python main.py` locally and perform the following test cases[cite: 2]:

1. Single Hit Accuracy & Score Increment:
   * Fire a single bullet (`Space`) at an isolated enemy[cite: 2, 6].
   * Expected: Bullet disappears upon hit, enemy disappears, score increments by exactly 1[cite: 2, 6].
2. Consecutive Rapid Bullets (Task 1 Regression Test):
   * Rapidly tap `Space` to launch multiple bullets in succession[cite: 2, 6].
   * Align player ship so consecutive bullets hit adjacent enemies[cite: 2, 6].
   * Expected: Every bullet that collides with an enemy registers a hit; zero bullets pass through active enemies[ci
3. Off-Screen Bullet Cleanup:
   * Fire bullets into empty space (where enemies have been destroyed)[cite: 2, 6].
   * Expected: Bullets move upward past  $y = 0$  and clean up without errors[cite: 4, 6].
4. General Mechanics:
   * Verify player movement (`A`/`D`/Arrows) and enemy movement remain unaffected[cite: 2, 6, 7].

---

### Automated Verification Script (`test_collision.py`)

You can optionally create and run this lightweight test script locally (`python test_collision.py`) to verify collision

```python
import os
import pygame

# Initialize dummy audio/video drivers for headless testing
os.environ["SDL_VIDEODRIVER"] = "dummy"
pygame.init()

```

```

pygame.display.set_mode((1, 1))

from game.game_engine import GameEngine
from game.bullet import Bullet

def test_consecutive_bullet_collisions():
    engine = GameEngine(600, 700)

    # Spawn two bullets directly positioned on top of two active enemies
    enemy1 = engine.enemy_grid.enemies[0]
    enemy2 = engine.enemy_grid.enemies[1]

    b1 = Bullet(enemy1.x + 2, enemy1.y + 2, direction=-1)
    b2 = Bullet(enemy2.x + 2, enemy2.y + 2, direction=-1)

    engine.player_bullets = [b1, b2]

    # Trigger engine update
    engine.update()

    # Assertions
    assert not enemy1.alive, "Enemy 1 should be destroyed"
    assert not enemy2.alive, "Enemy 2 should be destroyed"
    assert engine.score == 2, f"Score should be 2, got {engine.score}"
    assert len(engine.player_bullets) == 0, "Both bullets should be removed"
    print("ALL TASK 1 AUTOMATED COLLISION TESTS PASSED SUCCESSFULLY!")

if __name__ == "__main__":
    test_consecutive_bullet_collisions()
'''[cite: 1, 4, 5, 6]

---

### Next Steps

```

Please test the updated code locally. Once you confirm Task 1 is working and have recorded your "after" video segment fo

*User prompt: Review your Task 1 collision-detection implementation and the proposed test\_collision.py against the actual source files I uploaded. Before I apply the changes to my local repository, verify these specific points: Re-read the actual game/game\_engine.py, game/bullet.py, and game/enemy.py files. Do not rely on assumptions about their implementation. Verify that the bullets\_to\_remove set is compatible with the actual Bullet class. Confirm whether Bullet instances are hashable. If they are not, suggest the smallest safe correction. Review the proposed automated test carefully. In GameEngine.update(), bullets are moved before collision detection. The proposed test positions bullets at enemy.y + 2 before calling update(), so a bullet moving upward by 8 pixels may no longer overlap the enemy when the collision check occurs. Check this against the actual bullet dimensions and movement logic, and correct the test if necessary. Account for enemy movement during update() so the collision test does not depend on unpredictable positions. Also prevent random enemy firing or other game mechanics from making the test unreliable. Ensure the test genuinely reproduces the original mutation-during-iteration bug and verifies that two consecutive bullets can destroy two distinct enemies, with the score increasing exactly twice. Check that the proposed implementation does not introduce regressions involving repeated scoring, bullets hitting already-destroyed enemies, or safe bullet cleanup. Provide the corrected, complete test\_collision.py and the exact command to execute it. State whether the implementation itself needs any correction, explaining why. Do not implement Tasks 2, 3, or 4. Do not claim the test passed unless you actually executed it in an environment with the required source files and dependencies. If you cannot execute it, clearly distinguish your code review from an actual test result. Keep the solution minimal and appropriate for this college lab.*

## Response: Critical Source Code Review & Verification

### 1. Hashability of Bullet Instances

- **Inspection of game/bullet.py:** The Bullet class defines an explicit `__init__` constructor and method functions (`move`, `off_screen`, `rect`), but **does not** define custom `__eq__` or `__hash__` methods.
- **Python standard behavior:** In Python, custom classes that do not override `__eq__` inherit default object identity equality (`id(self)`) and the default `__hash__` method from `object`. Therefore, Bullet instances **are naturally hashable** by their object identity memory address and work cleanly inside a Python set without raising `TypeError: unhashable type`.
- **Conclusion for Task 1 Implementation:** The implementation using `bullets_to_remove = set()` is safe and fully compatible with Bullet.

### 2. Analysis of Test Logic & Edge Cases in GameEngine.update()

Re-examining `GameEngine.update()` line-by-line reveals several execution ordering details:

1. `self.enemy_grid.move()` runs first, which moves alive enemies horizontally by `self.speed` (1.5 pixels).
2. **Random enemy firing loop** runs second, which can spawn random downward enemy bullets.

3. `bullet.move()` runs third:

- Player bullets move upward ( $y$  decreases by 8 pixels).

4. **Collision check** runs fourth.

#### Discrepancy in the Previous Test Script:

- In the previous draft, bullets were placed at `enemy.y + 2` before calling `engine.update()`.
- When `engine.update()` ran, the bullet moved 8 pixels upward (`direction = -1, speed = 8`), placing its top edge at  $(\text{enemy.y} + 2) - 8 = \text{enemy.y} - 6$ .
- Since the bullet height is 12 pixels, its bounding rectangle spanned  $y \in [\text{enemy.y} - 6, \text{enemy.y} + 6]$ . Since the enemy bounding box spans  $y \in [\text{enemy.y}, \text{enemy.y} + 30]$ , `colliderect()` would still overlap.
- **However**, to eliminate reliance on frame movement timing, exact relative positioning, or random enemy firing (`enemy_fire_chance = 0.01`), a robust test should explicitly isolate collision handling.

### 3. Verification of Task 1 Collision Implementation

Re-evaluating the proposed collision handling block for `GameEngine.update()`:

```
# --- TASK 1: Refined Collision Detection ---
bullets_to_remove = set()
alive_enemies = self.enemy_grid.alive_enemies()

for bullet in self.player_bullets:
    for enemy in alive_enemies:
        if enemy.alive and bullet.rect().colliderect(enemy.rect()):
            enemy.alive = False
            bullets_to_remove.add(bullet)
            self.score += 1
            break # Stop checking this bullet against other enemies

if bullets_to_remove:
    self.player_bullets = [b for b in self.player_bullets if b not in bullets_to_remove]
```[cite: 6]

* **Does it cause repeated scoring or hit destroyed enemies?** No. The condition `if enemy.alive ...` prevents an already destroyed enemy from being hit again.
* **Does a single bullet score multiple times?** No. The `break` statement immediately exits the inner enemy loop once a collision is found.
* **Does it resolve skipped elements?** Yes. `self.player_bullets` is not modified during the loop execution, so Python's iteration order is preserved.
* **Is bullet cleanup safe?** Yes. All collided bullets are collected in `bullets_to_remove` and filtered out after the loop.

**Status:** The Task 1 implementation in `game/game_engine.py` is accurate and requires no further code modifications.

---

### Corrected Automated Test Script (`test_collision.py`)

```

This test script programmatically demonstrates the original bug and validates the fix without requiring a display or audio.

```
```python
import os
import pygame

# Run headlessly without opening a GUI window
os.environ["SDL_VIDEODRIVER"] = "dummy"
pygame.init()
pygame.display.set_mode((1, 1))

from game.game_engine import GameEngine
from game.bullet import Bullet

def test_original_bug_reproduction_and_fix():
    engine = GameEngine(600, 700)

    # Disable random enemy firing during testing to eliminate non-determinism
    engine.enemy_fire_chance = 0.0

    # Select two adjacent enemies from the grid
    enemy1 = engine.enemy_grid.enemies[0]
    enemy2 = engine.enemy_grid.enemies[1]

```

```

# Account for bullet movement during update():
# Bullet moves upward by 8 pixels in engine.update().
# Placing bullet.y at enemy.y + 15 ensures that after moving -8px (y = enemy.y + 7),
# the bullet (height 12) fully overlaps the enemy (height 30, y in [enemy.y, enemy.y + 30]).
b1 = Bullet(enemy1.x + 5, enemy1.y + 15, direction=-1)
b2 = Bullet(enemy2.x + 5, enemy2.y + 15, direction=-1)

# Place two consecutive bullets in player_bullets list
engine.player_bullets = [b1, b2]

initial_score = engine.score

# Execute single game frame update
engine.update()

# Assertions to confirm both collisions register correctly
assert not enemy1.alive, "Bug detected: Enemy 1 was not destroyed!"
assert not enemy2.alive, "Bug detected: Enemy 2 was skipped due to list mutation bug!"
assert engine.score == initial_score + 2, f"Score should increment by 2, got {engine.score}"
assert len(engine.player_bullets) == 0, f"Both bullets should be removed, remaining: {len(engine.player_bullets)}"

print("TASK 1 VERIFICATION SUCCESSFUL: Both consecutive collisions registered correctly!")

if __name__ == "__main__":
    test_original_bug_reproduction_and_fix()
'''[cite: 1, 4, 5, 6]

---

### Local Execution Command

Run the test script locally from your project root directory:

```bash
python test_collision.py
'''[cite: 2]

*Note on Execution:* As an AI partner without direct local system terminal execution access in this session, I have veri

```

---

*User prompt: The code you provided for Task 1 appears to contain Markdown formatting and citation markers that were accidentally copied into my Python source file. When I run python test\_collision.py, Python reports: SyntaxError: invalid syntax File "game\game\_engine.py", line 8 I need you to correct this carefully. 1. Re-examine the original game/game\_engine.py that I uploaded and your Task 1 implementation. 2. Return the complete corrected contents of game/game\_engine.py as valid, executable Python code. 3. The code must contain no Markdown fences, citation markers, explanatory prose, or non-Python text. 4. Preserve the original game functionality and apply only the Task 1 collision-detection fix. 5. Do not silently omit any original methods or change unrelated game mechanics. 6. After the code, provide a separate explanation outside the code block identifying the collision-handling changes. 7. Also tell me how to verify that the saved file contains only Python code. Do not implement Tasks 2, 3, or 4. Do not claim to have run the code unless you actually executed it.*

**Response:** Here is the clean, raw Python code for game/game\_engine.py. You can copy and paste this directly into your file without any syntax issues.

### game/game\_engine.py

```

import pygame
import random
from .player import Player
from .enemy import EnemyGrid
from .bullet import Bullet

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.player = Player(width // 2 - 20, height - 50, 40, 20)
        self.enemy_grid = EnemyGrid(width)

```

```

self.player_bullets = []
self.enemy_bullets = []
self._shoot_cooldown = 0
self.enemy_fire_chance = 0.01

self.score = 0
self.font = pygame.font.SysFont("Arial", 30)
self.game_over = False

def handle_event(self, event):
    if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
        if self._shoot_cooldown <= 0:
            bullet_x = self.player.center_x() - 2
            self.player_bullets.append(Bullet(bullet_x, self.player.y, direction=-1))
            self._shoot_cooldown = 15

def handle_input(self):
    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT] or keys[pygame.K_a]:
        self.player.move(-self.player.speed, self.width)
    if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
        self.player.move(self.player.speed, self.width)

def update(self):
    if self.game_over:
        return

    if self._shoot_cooldown > 0:
        self._shoot_cooldown -= 1

    self.enemy_grid.move()

    for enemy in self.enemy_grid.alive_enemies():
        if random.random() < self.enemy_fire_chance:
            bullet_x = enemy.x + enemy.width // 2
            self.enemy_bullets.append(Bullet(bullet_x, enemy.y + enemy.height, direction=1))

    for bullet in self.player_bullets:
        bullet.move()
    for bullet in self.enemy_bullets:
        bullet.move()

    self.player_bullets = [b for b in self.player_bullets if not b.off_screen(self.height)]
    self.enemy_bullets = [b for b in self.enemy_bullets if not b.off_screen(self.height)]

    bullets_to_remove = set()
    alive_enemies = self.enemy_grid.alive_enemies()

    for bullet in self.player_bullets:
        for enemy in alive_enemies:
            if enemy.alive and bullet.rect().colliderect(enemy.rect()):
                enemy.alive = False
                bullets_to_remove.add(bullet)
                self.score += 1
                break

    if bullets_to_remove:
        self.player_bullets = [b for b in self.player_bullets if b not in bullets_to_remove]

    for bullet in self.enemy_bullets:
        if bullet.rect().colliderect(self.player.rect()):
            self.game_over = True
            break

    if self.enemy_grid.reached_bottom(self.player.y):
        self.game_over = True

def render(self, screen):
    pygame.draw.rect(screen, GREEN, self.player.rect())

    for enemy in self.enemy_grid.alive_enemies():
        pygame.draw.rect(screen, WHITE, enemy.rect())

```

```

for bullet in self.player_bullets:
    pygame.draw.rect(screen, WHITE, bullet.rect())
for bullet in self.enemy_bullets:
    pygame.draw.rect(screen, RED, bullet.rect())

score_text = self.font.render(f"Score: {self.score}", True, WHITE)
screen.blit(score_text, (10, 10))

if self.game_over and not getattr(self, "_game_over_logged", False):
    print("Game over! Final score:", self.score)
    self._game_over_logged = True

```

## Summary of Changes in Collision Handling

- 1. Eliminated In-Place Mutation:** Instead of removing bullets directly inside the loop via `self.player_bullets.remove(bullet)`, colliding bullets are collected in `bullets_to_remove = set()`.
- 2. Immutable Iteration List:** The loop iterates through `self.player_bullets` without altering the list structure midway, preventing Python's iterator index from skipping adjacent elements.
- 3. Post-Loop Filtering:** After checking all collisions, `self.player_bullets` is updated via a list comprehension filtering out bullets present in `bullets_to_remove`.
- 4. Already-Destroyed Guard:** Added `enemy.alive` check (if `enemy.alive` and ...) to ensure destroyed enemies are not hit multiple times in a single pass.

## How to Verify the File Contains Only Valid Python Code

To verify your file is free of formatting or syntax errors before executing the full game:

- 1. Run Python's Built-in Syntax Checker:** In your terminal, run:

```
python -m py_compile game/game_engine.py
```

If it returns with no output and exits to the command prompt, the file contains zero syntax errors.

- 2. Run the Test Script:** Now run the test script locally:

```
python test_collision.py
```[cite: 2]
```

User prompt: We are working on Task 2 of my Space Invaders Pygame lab. Task 1, fixing player bullet collision detection, is complete and committed. Now implement ONLY Task 2: a proper Game Over screen. Project structure: `main.py game/game_engine.py game/player.py game/enemy.py game/bullet.py` Requirements: When the player loses or the game otherwise reaches its existing game-over condition, display a clear "GAME OVER" message inside the Pygame window. Display the final score prominently on the screen. Keep the window responsive while the game-over screen is visible. Continue processing Pygame events so the window can be closed normally. Stop normal gameplay updates after game over. Do not continue moving enemies, firing bullets, or changing the score. Keep the game-over screen displayed until the user closes the window. Do not implement replay, difficulty selection, or sound effects; those are later tasks. Preserve the existing project structure, controls, gameplay, and Task 1 collision fix. Follow the existing coding style and use the current architecture. Avoid unnecessary refactoring or new dependencies. Inspect the actual uploaded source code before making changes. Identify which files need modification, then provide the complete corrected contents of each modified file so I can replace them directly. Do not use Markdown code fences inside the code itself, citations, or explanatory text mixed into Python files. Clearly label each filename outside its code block. Ensure the game-over screen is drawn every frame after game over, and the event loop remains responsive. Avoid blocking calls such as `time.sleep()` or `input()`. Implement only Task 2. Briefly explain what changed and how I can test it.

Response: Here are the implementation details and full code updates for **Task 2: Game Over Screen**.

## Analysis & Files to Modify

To implement Task 2 while preserving the existing project architecture and Task 1 collision fix:

- 1. game/game\_engine.py:**

- Add a second, larger Pygame font (`self.large_font`) in `__init__` for rendering the "GAME OVER" header.
- In `render()`, when `self.game_over` is `True`, render a centered "GAME OVER" message and the final score text overlay on top of the screen instead of solely relying on `print()`.
- Continue to handle events and render the Game Over overlay smoothly every frame without blocking calls like `time.sleep()`.

- 2. main.py:**

- No modification is strictly necessary if event handling and display rendering loop continuously in `main.py` regardless of `self.game_over`. However, checking `main.py` confirms that the game loop already handles `QUIT` events and renders `engine.render(SCREEN)` every frame regardless of game state.

## Complete File Contents

game/game\_engine.py

```
import pygame
import random
from .player import Player
from .enemy import EnemyGrid
from .bullet import Bullet

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.player = Player(width // 2 - 20, height - 50, 40, 20)
        self.enemy_grid = EnemyGrid(width)

        self.player_bullets = []
        self.enemy_bullets = []
        self._shoot_cooldown = 0
        self.enemy_fire_chance = 0.01

        self.score = 0
        self.font = pygame.font.SysFont("Arial", 30)
        self.large_font = pygame.font.SysFont("Arial", 60)
        self.game_over = False

    def handle_event(self, event):
        if self.game_over:
            return

        if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
            if self._shoot_cooldown <= 0:
                bullet_x = self.player.center_x() - 2
                self.player_bullets.append(Bullet(bullet_x, self.player.y, direction=-1))
                self._shoot_cooldown = 15

    def handle_input(self):
        if self.game_over:
            return

        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            self.player.move(-self.player.speed, self.width)
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            self.player.move(self.player.speed, self.width)

    def update(self):
        if self.game_over:
            return

        if self._shoot_cooldown > 0:
            self._shoot_cooldown -= 1

        self.enemy_grid.move()

        for enemy in self.enemy_grid.alive_enemies():
            if random.random() < self.enemy_fire_chance:
                bullet_x = enemy.x + enemy.width // 2
                self.enemy_bullets.append(Bullet(bullet_x, enemy.y + enemy.height, direction=1))

        for bullet in self.player_bullets:
            bullet.move()
        for bullet in self.enemy_bullets:
            bullet.move()

        self.player_bullets = [b for b in self.player_bullets if not b.off_screen(self.height)]
```

```

self.enemy_bullets = [b for b in self.enemy_bullets if not b.off_screen(self.height)]

bullets_to_remove = set()
alive_enemies = self.enemy_grid.alive_enemies()

for bullet in self.player_bullets:
    for enemy in alive_enemies:
        if enemy.alive and bullet.rect().colliderect(enemy.rect()):
            enemy.alive = False
            bullets_to_remove.add(bullet)
            self.score += 1
            break

if bullets_to_remove:
    self.player_bullets = [b for b in self.player_bullets if b not in bullets_to_remove]

for bullet in self.enemy_bullets:
    if bullet.rect().colliderect(self.player.rect()):
        self.game_over = True
        break

if self.enemy_grid.reached_bottom(self.player.y):
    self.game_over = True

def render(self, screen):
    pygame.draw.rect(screen, GREEN, self.player.rect())

    for enemy in self.enemy_grid.alive_enemies():
        pygame.draw.rect(screen, WHITE, enemy.rect())

    for bullet in self.player_bullets:
        pygame.draw.rect(screen, WHITE, bullet.rect())
    for bullet in self.enemy_bullets:
        pygame.draw.rect(screen, RED, bullet.rect())

    score_text = self.font.render(f"Score: {self.score}", True, WHITE)
    screen.blit(score_text, (10, 10))

    if self.game_over:
        game_over_surface = self.large_font.render("GAME OVER", True, RED)
        final_score_surface = self.font.render(f"Final Score: {self.score}", True, WHITE)

        go_rect = game_over_surface.get_rect(center=(self.width // 2, self.height // 2 - 30))
        score_rect = final_score_surface.get_rect(center=(self.width // 2, self.height // 2 + 30))

        screen.blit(game_over_surface, go_rect)
        screen.blit(final_score_surface, score_rect)

```

## Summary of Changes

- Visual UI Overlay:** Created `self.large_font` in `GameEngine.__init__`. When `self.game_over` is set to `True`, `render()` calculates the center coordinates of the screen and renders centered text for "GAME OVER" (in RED) and "Final Score: X" (in WHITE).
- Input Isolation:** Added `if self.game_over: return` guards to `handle_event()` and `handle_input()` so player shooting and movement ignore user input once the game ends.
- Event Loop Continuity:** The main Pygame event loop in `main.py` continues running at full 60 FPS, maintaining full window responsiveness (such as clicking the window 'X' close button).

## How to Test Task 2 Locally

- Launch the game:

```
python main.py
```[cite: 2]
```

- Intentionally let a red enemy bullet hit your ship or let enemies reach the bottom baseline.
- Verify that:
  - Red "GAME OVER" text and your final score appear clearly in the middle of the game window.
  - Ship movement, bullet firing, and enemy movements freeze.
  - The game window remains active and can be closed cleanly via the 'X' button.

*User prompt: After your Task 2 changes, running python main.py fails with: ImportError: attempted relative import with no known parent package The traceback points to main.py, line 3: from .player import Player Our project structure is: main.py at the project root game/player.py game/game\_engine.py game/enemy.py game/bullet.py The required launch command is python main.py from the project root. Inspect the actual files and fix the imports so the project runs with that command. Preserve the existing package structure and Task 1 collision fix. Do not change gameplay behavior or implement any additional tasks. Return the complete corrected contents of every file that needs modification, with filenames clearly labeled outside the code blocks. Do not include citations or Markdown inside the Python code.*

Response: Here is the root cause analysis and the exact fix for the ImportError.

## Root Cause Analysis

When launching the application via `python main.py` from the project root directory, `main.py` is executed as the **top-level script** (`__name__ == "__main__"`).

In Python, top-level scripts do not have a known parent package context. Inside `game/game_engine.py`, the import statements were written using explicit relative imports (`from .player import Player`). When `main.py` imports `GameEngine` from `game.game_engine`, Python attempts to resolve those relative dots (`.player`, `.enemy`, `.bullet`) relative to the executing module's package structure, which fails with `ImportError: attempted relative import with no known parent package`.

To make `game/game_engine.py` compatible with standard execution via `python main.py`, the relative dots must be converted to package-qualified imports (`from game.player import Player`, `from game.enemy import EnemyGrid`, `from game.bullet import Bullet`).

## File Modifications

Only `game/game_engine.py` needs modification. No changes are required in `main.py` or any other file.

`game/game_engine.py`

```
import pygame
import random
from game.player import Player
from game.enemy import EnemyGrid
from game.bullet import Bullet

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.player = Player(width // 2 - 20, height - 50, 40, 20)
        self.enemy_grid = EnemyGrid(width)

        self.player_bullets = []
        self.enemy_bullets = []
        self._shoot_cooldown = 0
        self.enemy_fire_chance = 0.01

        self.score = 0
        self.font = pygame.font.SysFont("Arial", 30)
        self.large_font = pygame.font.SysFont("Arial", 60)
        self.game_over = False

    def handle_event(self, event):
        if self.game_over:
            return

        if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
            if self._shoot_cooldown <= 0:
                bullet_x = self.player.center_x() - 2
                self.player_bullets.append(Bullet(bullet_x, self.player.y, direction=-1))
                self._shoot_cooldown = 15

    def handle_input(self):
        if self.game_over:
            return
```

```

keys = pygame.key.get_pressed()
if keys[pygame.K_LEFT] or keys[pygame.K_a]:
    self.player.move(-self.player.speed, self.width)
if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
    self.player.move(self.player.speed, self.width)

def update(self):
    if self.game_over:
        return

    if self._shoot_cooldown > 0:
        self._shoot_cooldown -= 1

    self.enemy_grid.move()

    for enemy in self.enemy_grid.alive_enemies():
        if random.random() < self.enemy_fire_chance:
            bullet_x = enemy.x + enemy.width // 2
            self.enemy_bullets.append(Bullet(bullet_x, enemy.y + enemy.height, direction=1))

    for bullet in self.player_bullets:
        bullet.move()
    for bullet in self.enemy_bullets:
        bullet.move()

    self.player_bullets = [b for b in self.player_bullets if not b.off_screen(self.height)]
    self.enemy_bullets = [b for b in self.enemy_bullets if not b.off_screen(self.height)]

    bullets_to_remove = set()
    alive_enemies = self.enemy_grid.alive_enemies()

    for bullet in self.player_bullets:
        for enemy in alive_enemies:
            if enemy.alive and bullet.rect().colliderect(enemy.rect()):
                enemy.alive = False
                bullets_to_remove.add(bullet)
                self.score += 1
                break

    if bullets_to_remove:
        self.player_bullets = [b for b in self.player_bullets if b not in bullets_to_remove]

    for bullet in self.enemy_bullets:
        if bullet.rect().colliderect(self.player.rect()):
            self.game_over = True
            break

    if self.enemy_grid.reached_bottom(self.player.y):
        self.game_over = True

def render(self, screen):
    pygame.draw.rect(screen, GREEN, self.player.rect())

    for enemy in self.enemy_grid.alive_enemies():
        pygame.draw.rect(screen, WHITE, enemy.rect())

    for bullet in self.player_bullets:
        pygame.draw.rect(screen, WHITE, bullet.rect())
    for bullet in self.enemy_bullets:
        pygame.draw.rect(screen, RED, bullet.rect())

    score_text = self.font.render(f"Score: {self.score}", True, WHITE)
    screen.blit(score_text, (10, 10))

    if self.game_over:
        game_over_surface = self.large_font.render("GAME OVER", True, RED)
        final_score_surface = self.font.render(f"Final Score: {self.score}", True, WHITE)

        go_rect = game_over_surface.get_rect(center=(self.width // 2, self.height // 2 - 30))
        score_rect = final_score_surface.get_rect(center=(self.width // 2, self.height // 2 + 30))

        screen.blit(game_over_surface, go_rect)
        screen.blit(final_score_surface, score_rect)

```

## Verification Command

You can now run the game from the project root directory:

```
python main.py
```[cite: 2]
```

---

*User prompt: Implement Task 3 for my existing Space Invaders Pygame lab project. Task 1 (collision detection) and Task 2 (Game Over screen with final score) are already completed and pushed to GitHub. Project structure: main.py game/game\_engine.py game/player.py game/enemy.py game/bullet.py Requirements: After Game Over, show options to play again or exit. Allow the player to select Easy, Medium, or Hard difficulty before starting or restarting a game. Difficulty must affect gameplay speed and enemy firing rate: Easy: slower enemies and a lower enemy firing rate. Medium: default/balanced speeds and firing rate. Hard: faster enemies and a higher enemy firing rate. Show the choices clearly on screen and allow selection using keyboard controls. Display the selected difficulty. Choosing Play Again should reset the score, player position, enemies, bullets, and game-over state so a new game starts cleanly at the selected difficulty. Choosing Exit or closing the window should quit gracefully. Preserve Task 1 collision detection and Task 2 Game Over screen. Keep the final score visible on the Game Over screen. Keep the current project structure, controls, and dependencies. Do not add sound effects yet; those belong to Task 4. Inspect the actual source code before editing. Make only necessary changes, and return the complete contents of each modified file, clearly labeling filenames outside the code blocks. Do not include citations, Markdown fences, or explanations inside the Python files. Ensure the game remains responsive and does not use blocking calls such as input() or time.sleep(). Implement only Task 3. Briefly explain how to test the new replay and difficulty features.*

Response: Here are the complete file updates for **Task 3: Add Replay Option with Difficulty Settings**.

## Analysis & Modification Strategy

To implement Task 3 cleanly while maintaining project structure and existing features:

1. **game/enemy.py (EnemyGrid):**
  - Update EnemyGrid.\_\_init\_\_ to accept speed as a parameter (defaulting to 1.5 if omitted).
2. **game/game\_engine.py:**
  - Add a state machine or UI menu state (START\_MENU, PLAYING, GAME\_OVER).
  - Support three difficulty presets (Easy, Medium, Hard) mapping to distinct enemy speed and fire chance values:
    - **Easy:** speed = 1.0, fire\_chance = 0.005
    - **Medium:** speed = 1.5, fire\_chance = 0.01
    - **Hard:** speed = 2.5, fire\_chance = 0.02
  - Render difficulty selection options on the main start screen and game over screen.
  - Add a reset\_game(difficulty) method that cleans up all existing state (score, player position, bullets, enemy grid) and restarts gameplay.
  - Handle key events (1/2/3 or Arrow keys / Enter / R / Q) on the menu and game-over screens.

## Modified Files

game/enemy.py

```
import pygame
```

```
class Enemy:
    def __init__(self, x, y, width=40, height=30):
        self.x = x
        self.y = y
        self.width = width
        self.height = height
        self.alive = True

    def rect(self):
        return pygame.Rect(self.x, self.y, self.width, self.height)
```

```
class EnemyGrid:
    def __init__(self, screen_width, rows=4, cols=8, speed=1.5, drop_amount=20):
        self.screen_width = screen_width
        self.speed = speed
        self.drop_amount = drop_amount
        self.direction = 1 # 1 = moving right, -1 = moving left
```

```

        self.enemies = []
        spacing_x, spacing_y = 55, 45
        start_x, start_y = 40, 40
        for row in range(rows):
            for col in range(cols):
                x = start_x + col * spacing_x
                y = start_y + row * spacing_y
                self.enemies.append(Enemy(x, y))

    def alive_enemies(self):
        return [e for e in self.enemies if e.alive]

    def move(self):
        alive = self.alive_enemies()
        if not alive:
            return

        min_x = min(e.x for e in alive)
        max_x = max(e.x + e.width for e in alive)

        hit_edge = (self.direction == 1 and max_x + self.speed >= self.screen_width) or \
            (self.direction == -1 and min_x - self.speed <= 0)

        if hit_edge:
            self.direction *= -1
            for e in alive:
                e.y += self.drop_amount
        else:
            for e in alive:
                e.x += self.speed * self.direction

    def reached_bottom(self, limit_y):
        return any(e.y + e.height >= limit_y for e in self.alive_enemies())

```

#### game/game\_engine.py

```

import pygame
import random

from game.player import Player
from game.enemy import EnemyGrid
from game.bullet import Bullet

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)
YELLOW = (255, 255, 0)

# Difficulty settings: (speed, fire_chance)
DIFFICULTIES = {
    "Easy": (1.0, 0.005),
    "Medium": (1.5, 0.01),
    "Hard": (2.5, 0.02)
}

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.font = pygame.font.SysFont("Arial", 26)
        self.large_font = pygame.font.SysFont("Arial", 50)
        self.small_font = pygame.font.SysFont("Arial", 20)

        self.current_difficulty = "Medium"
        self.state = "MENU" # States: MENU, PLAYING, GAME_OVER
        self.game_over = False

        self.reset_game(self.current_difficulty)

    def reset_game(self, difficulty="Medium"):
        self.current_difficulty = difficulty

```

```

speed, fire_chance = DIFFICULTIES[difficulty]

self.player = Player(self.width // 2 - 20, self.height - 50, 40, 20)
self.enemy_grid = EnemyGrid(self.width, speed=speed)

self.player_bullets = []
self.enemy_bullets = []
self._shoot_cooldown = 0
self.enemy_fire_chance = fire_chance

self.score = 0
self.game_over = False

def handle_event(self, event):
    if event.type != pygame.KEYDOWN:
        return

    if self.state in ("MENU", "GAME_OVER"):
        if event.key == pygame.K_1:
            self.reset_game("Easy")
            self.state = "PLAYING"
        elif event.key == pygame.K_2:
            self.reset_game("Medium")
            self.state = "PLAYING"
        elif event.key == pygame.K_3:
            self.reset_game("Hard")
            self.state = "PLAYING"
        elif event.key in (pygame.K_RETURN, pygame.K_SPACE):
            self.reset_game(self.current_difficulty)
            self.state = "PLAYING"
        elif event.key == pygame.K_q:
            pygame.event.post(pygame.event.Event(pygame.QUIT))

    elif self.state == "PLAYING":
        if event.key == pygame.K_SPACE and self._shoot_cooldown <= 0:
            bullet_x = self.player.center_x() - 2
            self.player_bullets.append(Bullet(bullet_x, self.player.y, direction=-1))
            self._shoot_cooldown = 15

def handle_input(self):
    if self.state != "PLAYING":
        return

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT] or keys[pygame.K_a]:
        self.player.move(-self.player.speed, self.width)
    if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
        self.player.move(self.player.speed, self.width)

def update(self):
    if self.state != "PLAYING":
        return

    if self._shoot_cooldown > 0:
        self._shoot_cooldown -= 1

    self.enemy_grid.move()

    for enemy in self.enemy_grid.alive_enemies():
        if random.random() < self.enemy_fire_chance:
            bullet_x = enemy.x + enemy.width // 2
            self.enemy_bullets.append(Bullet(bullet_x, enemy.y + enemy.height, direction=1))

    for bullet in self.player_bullets:
        bullet.move()
    for bullet in self.enemy_bullets:
        bullet.move()

    self.player_bullets = [b for b in self.player_bullets if not b.off_screen(self.height)]
    self.enemy_bullets = [b for b in self.enemy_bullets if not b.off_screen(self.height)]

    bullets_to_remove = set()
    alive_enemies = self.enemy_grid.alive_enemies()

```

```

for bullet in self.player_bullets:
    for enemy in alive_enemies:
        if enemy.alive and bullet.rect().collidect(enemy.rect()):
            enemy.alive = False
            bullets_to_remove.add(bullet)
            self.score += 1
            break

if bullets_to_remove:
    self.player_bullets = [b for b in self.player_bullets if b not in bullets_to_remove]

for bullet in self.enemy_bullets:
    if bullet.rect().collidect(self.player.rect()):
        self.game_over = True
        self.state = "GAME_OVER"
        break

if self.enemy_grid.reached_bottom(self.player.y):
    self.game_over = True
    self.state = "GAME_OVER"

def render(self, screen):
    if self.state == "MENU":
        title = self.large_font.render("SPACE INVADERS", True, GREEN)
        sub = self.font.render("Select Difficulty to Start:", True, WHITE)

        d1 = self.font.render("1 - Easy", True, WHITE if self.current_difficulty != "Easy" else YELLOW)
        d2 = self.font.render("2 - Medium", True, WHITE if self.current_difficulty != "Medium" else YELLOW)
        d3 = self.font.render("3 - Hard", True, WHITE if self.current_difficulty != "Hard" else YELLOW)

        start_txt = self.small_font.render("Press SPACE or RETURN to Start Current Selection", True, WHITE)
        quit_txt = self.small_font.render("Press Q to Quit", True, WHITE)

        screen.blit(title, title.get_rect(center=(self.width // 2, 180)))
        screen.blit(sub, sub.get_rect(center=(self.width // 2, 260)))

        screen.blit(d1, d1.get_rect(center=(self.width // 2, 320)))
        screen.blit(d2, d2.get_rect(center=(self.width // 2, 360)))
        screen.blit(d3, d3.get_rect(center=(self.width // 2, 400)))

        screen.blit(start_txt, start_txt.get_rect(center=(self.width // 2, 480)))
        screen.blit(quit_txt, quit_txt.get_rect(center=(self.width // 2, 520)))
        return

    # Render active gameplay
    pygame.draw.rect(screen, GREEN, self.player.rect())

    for enemy in self.enemy_grid.alive_enemies():
        pygame.draw.rect(screen, WHITE, enemy.rect())

    for bullet in self.player_bullets:
        pygame.draw.rect(screen, WHITE, bullet.rect())
    for bullet in self.enemy_bullets:
        pygame.draw.rect(screen, RED, bullet.rect())

    score_text = self.font.render(f"Score: {self.score}", True, WHITE)
    diff_text = self.small_font.render(f"Difficulty: {self.current_difficulty}", True, YELLOW)
    screen.blit(score_text, (10, 10))
    screen.blit(diff_text, (self.width - 150, 15))

    if self.state == "GAME_OVER":
        go_surf = self.large_font.render("GAME OVER", True, RED)
        score_surf = self.font.render(f"Final Score: {self.score}", True, WHITE)

        prompt1 = self.small_font.render("Select Difficulty to Play Again:", True, WHITE)
        opts = self.small_font.render("1: Easy | 2: Medium | 3: Hard", True, YELLOW)
        prompt2 = self.small_font.render("Press SPACE to Restart Current | Q to Quit", True, WHITE)

        screen.blit(go_surf, go_surf.get_rect(center=(self.width // 2, self.height // 2 - 100)))
        screen.blit(score_surf, score_surf.get_rect(center=(self.width // 2, self.height // 2 - 40)))

        screen.blit(prompt1, prompt1.get_rect(center=(self.width // 2, self.height // 2 + 30)))
        screen.blit(opts, opts.get_rect(center=(self.width // 2, self.height // 2 + 60)))
        screen.blit(prompt2, prompt2.get_rect(center=(self.width // 2, self.height // 2 + 100)))

```

## Key Technical Changes

1. **Explicit Parameterized EnemyGrid Initialization:** Added `speed=1.5` parameter in `EnemyGrid.__init__()`.
2. **State Machine (MENU, PLAYING, GAME\_OVER):** Prevents gameplay updates from firing on menu/game-over screens while keeping the event loop responsive.
3. **Difficulty Presets:**
  - Key 1 → Easy (Speed 1.0, Fire Chance 0.005)
  - Key 2 → Medium (Speed 1.5, Fire Chance 0.01)
  - Key 3 → Hard (Speed 2.5, Fire Chance 0.02)
4. **Clean Replay Reset (reset\_game):** Re-instantiates Player, EnemyGrid, bullet lists, and resets score back to 0.
5. **Key Controls:** Added key listeners for 1/2/3, Space/Enter to replay, and Q to quit cleanly.

## How to Test Task 3 Locally

1. **Launch Game:** Run `python main.py`.
2. **Start Screen:** Verify the main screen displays the difficulty menu (1 - Easy, 2 - Medium, 3 - Hard).
3. **Select Difficulty:** Press 1, 2, or 3 to start playing at that difficulty. Confirm speed and enemy firing rate scale noticeably between Easy and Hard.
4. **Game Over & Replay:** Let an enemy hit you to trigger Game Over. Verify options appear to press 1, 2, 3 or Space to replay, or Q to exit.
5. **Score & State Reset:** Press 1 or Space after Game Over to confirm the score resets to 0, enemies reset to the top, and bullets clear.

User prompt: Implement Task 4 only for my existing Space Invaders Pygame lab project. Tasks 1–3 are complete: collision detection, Game Over screen with final score, and replay with Easy/Medium/Hard difficulty selection. Project structure: `main.py` `game/game_engine.py` `game/player.py` `game/enemy.py` `game/bullet.py` Add these three sound effects: Player firing: Play a short sound when the player fires a bullet. Enemy destroyed: Play a sound when a player bullet destroys an enemy. Game over: Play a sound when the game transitions into the Game Over state. Requirements: Use Pygame's built-in mixer (`pygame.mixer`) for audio playback. Prefer small, locally stored .wav sound files. If sound assets need to be created or sourced, explain exactly where to place them and ensure the project works without internet access at runtime. Initialize audio safely. If audio initialization or playback fails because sound is unavailable, the game must continue working without crashing. Avoid replaying the Game Over sound every frame; play it once per transition into Game Over. Avoid playing the enemy-destroyed sound for misses or repeated collision checks. Preserve all existing gameplay, collision detection, Game Over display, replay behavior, difficulty settings, and keyboard controls. Do not add new gameplay features or unnecessary dependencies. Inspect the actual uploaded source code before editing. Return complete contents of every modified file, with each filename clearly labeled outside its code block. Do not include citations or explanatory text inside the Python files. Implement only Task 4. Briefly explain how to test the sounds and where sound files should be placed.

Response: Here are the implementation details and full code updates for **Task 4: Sound Effects**.

## Analysis & Implementation Strategy

### 1. Audio Manager & Robust Fallbacks:

- Pygame's `pygame.mixer` requires .wav or .ogg sound files.
- Since no .wav files were included in the original repository, we create a fallback audio loader inside `game/game_engine.py`.
- If sound files exist in an `assets/sounds/` folder, Pygame loads and plays them.
- If sound files are missing or the system has no audio driver active, the game catches the exception gracefully and uses dummy/silent fallback objects so the game **never crashes**.

### 2. Sound Triggers:

- **Player Firing:** Triggered in `handle_event()` when a bullet is successfully spawned.
- **Enemy Destroyed:** Triggered in `update()` inside the collision loop when `enemy.alive` is set to `False`.
- **Game Over:** Triggered exactly once when transitioning `self.state` into `"GAME_OVER"`.

## Sound File Directory Structure

Place your sound files in an `assets/sounds/` directory inside your project root:

```
space-invaders-main/
├── assets/
│   └── sounds/
```

```

    |   |   |
    |   |   | laser.wav
    |   |   | explosion.wav
    |   |   | game_over.wav
    |   |
    |   | game/
    |   | |
    |   | | game_engine.py
    |   | | enemy.py
    |   | | player.py
    |   | | bullet.py
    |   |
    |   | main.py
    |
    | ``[cite: 1, 2]

```

```

---

### Modified Files

Only ``game/game_engine.py`` needs modification[cite: 6]. No changes are required in `main.py`, `enemy.py`, `player.py`

#### `game/game_engine.py`

```python
import os
import pygame
import random

from game.player import Player
from game.enemy import EnemyGrid
from game.bullet import Bullet

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)
YELLOW = (255, 255, 0)

# Difficulty settings: (speed, fire_chance)
DIFFICULTIES = {
    "Easy": (1.0, 0.005),
    "Medium": (1.5, 0.01),
    "Hard": (2.5, 0.02)
}

class DummySound:
    """Fallback object if audio files or mixer are unavailable."""
    def play(self):
        pass

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.font = pygame.font.SysFont("Arial", 26)
        self.large_font = pygame.font.SysFont("Arial", 50)
        self.small_font = pygame.font.SysFont("Arial", 20)

        self._init_sounds()

        self.current_difficulty = "Medium"
        self.state = "MENU"
        self.game_over = False

        self.reset_game(self.current_difficulty)

    def _init_sounds(self):
        """Safely initialize Pygame mixer and load sound assets with fallback."""
        self.sounds = {
            "shoot": DummySound(),
            "destroy": DummySound(),
            "game_over": DummySound()
        }

        try:
            if not pygame.mixer.get_init():

```

```

pygame.mixer.init()

sounds_dir = os.path.join("assets", "sounds")

shoot_path = os.path.join(sounds_dir, "laser.wav")
destroy_path = os.path.join(sounds_dir, "explosion.wav")
game_over_path = os.path.join(sounds_dir, "game_over.wav")

if os.path.exists(shoot_path):
    self.sounds["shoot"] = pygame.mixer.Sound(shoot_path)
if os.path.exists(destroy_path):
    self.sounds["destroy"] = pygame.mixer.Sound(destroy_path)
if os.path.exists(game_over_path):
    self.sounds["game_over"] = pygame.mixer.Sound(game_over_path)

except Exception:
    # If mixer or audio system fails, use DummySound fallbacks
    pass

def reset_game(self, difficulty="Medium"):
    self.current_difficulty = difficulty
    speed, fire_chance = DIFFICULTIES[difficulty]

    self.player = Player(self.width // 2 - 20, self.height - 50, 40, 20)
    self.enemy_grid = EnemyGrid(self.width, speed=speed)

    self.player_bullets = []
    self.enemy_bullets = []
    self._shoot_cooldown = 0
    self.enemy_fire_chance = fire_chance

    self.score = 0
    self.game_over = False

def trigger_game_over(self):
    if not self.game_over:
        self.game_over = True
        self.state = "GAME_OVER"
        self.sounds["game_over"].play()

def handle_event(self, event):
    if event.type != pygame.KEYDOWN:
        return

    if self.state in ("MENU", "GAME_OVER"):
        if event.key == pygame.K_1:
            self.reset_game("Easy")
            self.state = "PLAYING"
        elif event.key == pygame.K_2:
            self.reset_game("Medium")
            self.state = "PLAYING"
        elif event.key == pygame.K_3:
            self.reset_game("Hard")
            self.state = "PLAYING"
        elif event.key in (pygame.K_RETURN, pygame.K_SPACE):
            self.reset_game(self.current_difficulty)
            self.state = "PLAYING"
        elif event.key == pygame.K_q:
            pygame.event.post(pygame.event.Event(pygame.QUIT))

    elif self.state == "PLAYING":
        if event.key == pygame.K_SPACE and self._shoot_cooldown <= 0:
            bullet_x = self.player.center_x() - 2
            self.player_bullets.append(Bullet(bullet_x, self.player.y, direction=-1))
            self._shoot_cooldown = 15
            self.sounds["shoot"].play()

def handle_input(self):
    if self.state != "PLAYING":
        return

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT] or keys[pygame.K_a]:
        self.player.move(-self.player.speed, self.width)

```

```

    if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
        self.player.move(self.player.speed, self.width)

def update(self):
    if self.state != "PLAYING":
        return

    if self._shoot_cooldown > 0:
        self._shoot_cooldown -= 1

    self.enemy_grid.move()

    for enemy in self.enemy_grid.alive_enemies():
        if random.random() < self.enemy_fire_chance:
            bullet_x = enemy.x + enemy.width // 2
            self.enemy_bullets.append(Bullet(bullet_x, enemy.y + enemy.height, direction=1))

    for bullet in self.player_bullets:
        bullet.move()
    for bullet in self.enemy_bullets:
        bullet.move()

    self.player_bullets = [b for b in self.player_bullets if not b.off_screen(self.height)]
    self.enemy_bullets = [b for b in self.enemy_bullets if not b.off_screen(self.height)]

    bullets_to_remove = set()
    alive_enemies = self.enemy_grid.alive_enemies()

    for bullet in self.player_bullets:
        for enemy in alive_enemies:
            if enemy.alive and bullet.rect().colliderect(enemy.rect()):
                enemy.alive = False
                bullets_to_remove.add(bullet)
                self.score += 1
                self.sounds["destroy"].play()
                break

    if bullets_to_remove:
        self.player_bullets = [b for b in self.player_bullets if b not in bullets_to_remove]

    for bullet in self.enemy_bullets:
        if bullet.rect().colliderect(self.player.rect()):
            self.trigger_game_over()
            break

    if self.enemy_grid.reached_bottom(self.player.y):
        self.trigger_game_over()

def render(self, screen):
    if self.state == "MENU":
        title = self.large_font.render("SPACE INVADERS", True, GREEN)
        sub = self.font.render("Select Difficulty to Start:", True, WHITE)

        d1 = self.font.render("1 - Easy", True, WHITE if self.current_difficulty != "Easy" else YELLOW)
        d2 = self.font.render("2 - Medium", True, WHITE if self.current_difficulty != "Medium" else YELLOW)
        d3 = self.font.render("3 - Hard", True, WHITE if self.current_difficulty != "Hard" else YELLOW)

        start_txt = self.small_font.render("Press SPACE or RETURN to Start Current Selection", True, WHITE)
        quit_txt = self.small_font.render("Press Q to Quit", True, WHITE)

        screen.blit(title, title.get_rect(center=(self.width // 2, 180)))
        screen.blit(sub, sub.get_rect(center=(self.width // 2, 260)))

        screen.blit(d1, d1.get_rect(center=(self.width // 2, 320)))
        screen.blit(d2, d2.get_rect(center=(self.width // 2, 360)))
        screen.blit(d3, d3.get_rect(center=(self.width // 2, 400)))

        screen.blit(start_txt, start_txt.get_rect(center=(self.width // 2, 480)))
        screen.blit(quit_txt, quit_txt.get_rect(center=(self.width // 2, 520)))
        return

    pygame.draw.rect(screen, GREEN, self.player.rect())

    for enemy in self.enemy_grid.alive_enemies():

```

```

pygame.draw.rect(screen, WHITE, enemy.rect())

for bullet in self.player_bullets:
    pygame.draw.rect(screen, WHITE, bullet.rect())
for bullet in self.enemy_bullets:
    pygame.draw.rect(screen, RED, bullet.rect())

score_text = self.font.render(f"Score: {self.score}", True, WHITE)
diff_text = self.small_font.render(f"Difficulty: {self.current_difficulty}", True, YELLOW)
screen.blit(score_text, (10, 10))
screen.blit(diff_text, (self.width - 150, 15))

if self.state == "GAME_OVER":
    go_surf = self.large_font.render("GAME OVER", True, RED)
    score_surf = self.font.render(f"Final Score: {self.score}", True, WHITE)

    prompt1 = self.small_font.render("Select Difficulty to Play Again:", True, WHITE)
    opts = self.small_font.render("1: Easy | 2: Medium | 3: Hard", True, YELLOW)
    prompt2 = self.small_font.render("Press SPACE to Restart Current | Q to Quit", True, WHITE)

    screen.blit(go_surf, go_surf.get_rect(center=(self.width // 2, self.height // 2 - 100)))
    screen.blit(score_surf, score_surf.get_rect(center=(self.width // 2, self.height // 2 - 40)))

    screen.blit(prompt1, prompt1.get_rect(center=(self.width // 2, self.height // 2 + 30)))
    screen.blit(opts, opts.get_rect(center=(self.width // 2, self.height // 2 + 60)))
    screen.blit(prompt2, prompt2.get_rect(center=(self.width // 2, self.height // 2 + 100)))

```

## How to Test Task 4 Sounds

1. Create directory structure assets/sounds/ in your project folder.
  2. Add short .wav files named laser.wav, explosion.wav, and game\_over.wav.
  3. Run python main.py and test:
    - **Firing Sound:** Press Space to shoot. Confirm laser.wav plays once per bullet shot.
    - **Destroy Sound:** Destroy an enemy ship with a bullet. Confirm explosion.wav plays once per destroyed enemy.
    - **Game Over Sound:** Let an enemy bullet hit your ship. Confirm game\_over.wav plays **exactly once** when transitioning to the Game Over screen (and does not repeat every frame).
-